

1. TCP Server – Armstrong Number

ArmstrongServer.java

```
import java.net.*;
import java.io.*;

public class ArmstrongServer {
    public static void main(String[] args) throws Exception {
        ServerSocket ss = new ServerSocket(5000);

        System.out.println("Server started");
        System.out.println("Waiting for client...");

        Socket s = ss.accept();

        DataInputStream in = new DataInputStream(s.getInputStream());
        DataOutputStream out = new DataOutputStream(s.getOutputStream());

        int n = in.readInt();

        int temp = n;
        int sum = 0;
        int digits = 0;

        while (temp > 0) {
            digits++;
            temp = temp / 10;
        }

        temp = n;

        while (temp > 0) {
            int digit = temp % 10;
            int power = 1;

            for (int i = 1; i <= digits; i++) {
                power = power * digit;
            }

            sum = sum + power;
            temp = temp / 10;
        }

        if (sum == n)
            out.writeUTF(n + " is an Armstrong number");
        else
            out.writeUTF(n + " is not an Armstrong number");

        s.close();
        ss.close();
    }
}
```

ArmstrongClient.java

```
import java.net.*;
import java.io.*;
import java.util.Scanner;

public class ArmstrongClient {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);

        Socket s = new Socket("localhost", 5000);

        DataOutputStream out = new DataOutputStream(s.getOutputStream());
        DataInputStream in = new DataInputStream(s.getInputStream());

        System.out.print("Enter a number: ");
        int n = sc.nextInt();

        out.writeInt(n);

        String result = in.readUTF();

        System.out.println(result);

        s.close();
    }
}
```

Run the server first:

```
javac ArmstrongServer.java
java ArmstrongServer
```

Then run the client:

```
javac ArmstrongClient.java
java ArmstrongClient
```

2. UDP Client/Server – Exchange String Messages

UDPServer.java

```
import java.net.*;
import java.util.Scanner;

public class UDPServer {
    public static void main(String[] args) throws Exception {
        DatagramSocket socket = new DatagramSocket(5000);
        Scanner sc = new Scanner(System.in);

        byte data[] = new byte[1024];

        DatagramPacket packet = new DatagramPacket(data, data.length);
```

```

        System.out.println("Server waiting...");

        socket.receive(packet);

        String message = new String(packet.getData(), 0, packet.getLength());

        System.out.println("Client: " + message);

        System.out.print("Server: ");
        String reply = sc.nextLine();

        byte sendData[] = reply.getBytes();

        DatagramPacket sendPacket = new DatagramPacket(
            sendData,
            sendData.length,
            packet.getAddress(),
            packet.getPort()
        );

        socket.send(sendPacket);

        socket.close();
    }
}

```

UDPClient.java

```

import java.net.*;
import java.util.Scanner;

public class UDPClient {
    public static void main(String[] args) throws Exception {
        DatagramSocket socket = new DatagramSocket();
        Scanner sc = new Scanner(System.in);

        System.out.print("Client: ");
        String message = sc.nextLine();

        byte data[] = message.getBytes();

        InetAddress address = InetAddress.getByName("localhost");

        DatagramPacket packet = new DatagramPacket(
            data,
            data.length,
            address,
            5000
        );

        socket.send(packet);

        byte receiveData[] = new byte[1024];

        DatagramPacket receivePacket =

```

```

        new DatagramPacket(receiveData, receiveData.length);

socket.receive(receivePacket);

String reply = new String(
    receivePacket.getData(),
    0,
    receivePacket.getLength()
);

System.out.println("Server: " + reply);

socket.close();
    }
}

```

3. Java URL Class – Access Resource Information

```

import java.net.URL;

public class URLEDemo {
    public static void main(String[] args) throws Exception {
        URL url = new URL("https://www.google.com/index.html");

        System.out.println("Protocol: " + url.getProtocol());
        System.out.println("Host: " + url.getHost());
        System.out.println("Port: " + url.getPort());
        System.out.println("Path: " + url.getPath());
        System.out.println("File: " + url.getFile());
        System.out.println("URL: " + url);
    }
}

```

4. GUI Based Message Exchange Using Server and Client

Server.java

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.net.*;
import java.io.*;

public class Server extends JFrame {
    JTextField text;
    JButton send;
    Socket socket;
    DataOutputStream out;

    Server() {
        setTitle("Server");
        setSize(400, 150);
        setLayout(new FlowLayout());
    }
}

```

```

        text = new JTextField(20);
        send = new JButton("Send");

        add(text);
        add(send);

        try {
            ServerSocket ss = new ServerSocket(5000);
            socket = ss.accept();

            out = new DataOutputStream(socket.getOutputStream());

        } catch (Exception e) {
            System.out.println(e);
        }

        send.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                try {
                    out.writeUTF(text.getText());
                    text.setText("");
                } catch (Exception ex) {
                    System.out.println(ex);
                }
            }
        });

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }

    public static void main(String[] args) {
        new Server();
    }
}

```

Client.java

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.net.*;
import java.io.*;

public class Client extends JFrame {
    JTextField text;
    JButton send;
    Socket socket;
    DataOutputStream out;

    Client() {
        setTitle("Client");
        setSize(400, 150);
        setLayout(new FlowLayout());
    }
}

```

```

text = new JTextField(20);
send = new JButton("Send");

add(text);
add(send);

try {
    socket = new Socket("localhost", 5000);
    out = new DataOutputStream(socket.getOutputStream());

} catch (Exception e) {
    System.out.println(e);
}

send.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        try {
            out.writeUTF(text.getText());
            text.setText("");
        } catch (Exception ex) {
            System.out.println(ex);
        }
    }
});

setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setVisible(true);
}

public static void main(String[] args) {
    new Client();
}
}

```

To run Program 4

First run:

```

javac Server.java
java Server

```

Then in another terminal:

```

javac Client.java
java Client

```